

## 6교시: PostgreSQL container 실행

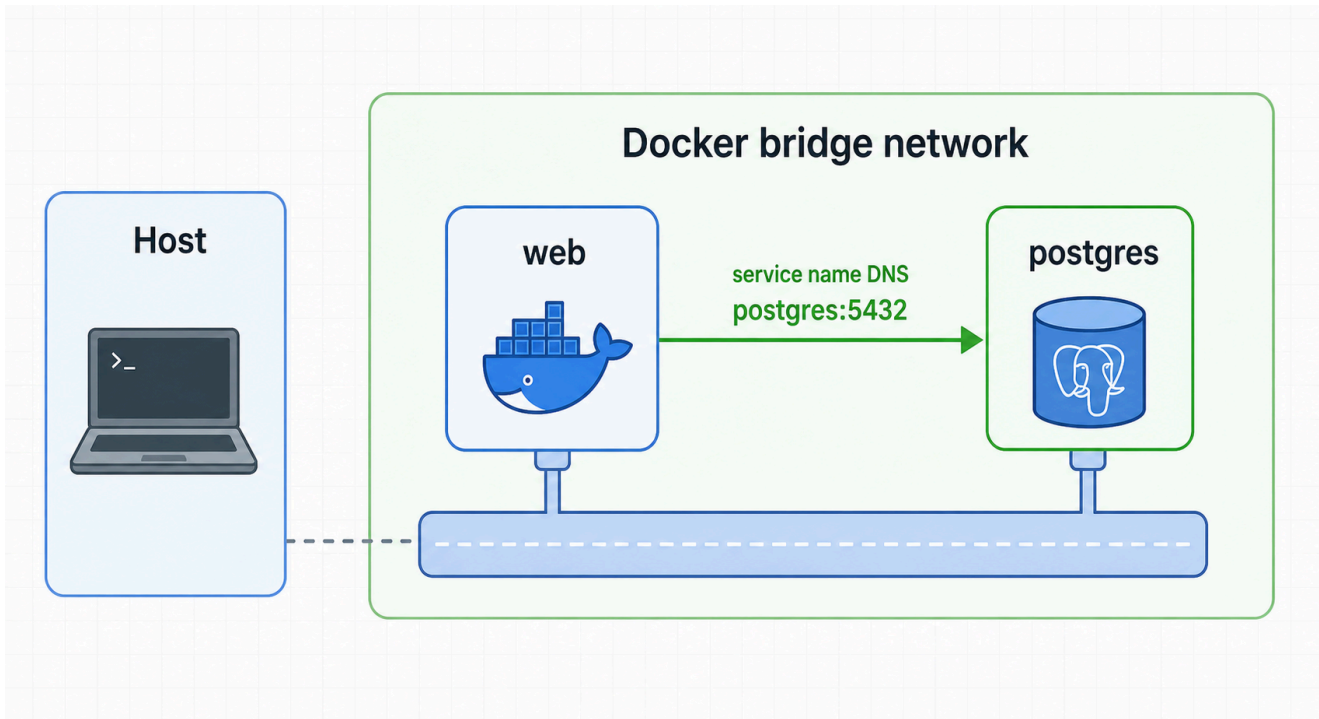
### 수업 목표

- DB container가 요구하는 environment, network, volume 조건을 구성한다.
- pg\_isready, psql, docker logs로 readiness evidence를 확보한다.
- DB를 host에 publish하지 않고 internal network에서 확인하는 구조를 이해한다.

### 50분 흐름

| 시간     | 활동                    | 비중     | 산출                 |
|--------|-----------------------|--------|--------------------|
| 0-8분   | DB container 실행 조건 정리 | 설명 15% | run option map     |
| 8-20분  | network/volume/env 준비 | 실행 25% | setup evidence     |
| 20-32분 | PostgreSQL 실행         | 실행 25% | container evidence |
| 32-42분 | readiness와 SQL 확인     | 실행 20% | DB evidence        |
| 42-50분 | logs와 운영 해석           | 설명 15% | log note           |

### Visual 1: DB runtime contract



PostgreSQL container는 network, environment, volume을 모두 요구하는 좋은 실습 대상이다. 이 이미지는 DB가 network 내부에서 service name으로 접근되는 흐름을 보여준다.

## 핵심 설명

DB container는 단순히 image를 실행하는 것만으로 충분하지 않다. 초기 password, database name, data directory, network membership을 함께 지정해야 한다.

postgres:16-alpine image는 POSTGRES\_PASSWORD 없이 초기화되지 않은 DB를 시작하면 실패한다. 이 실패는 Docker 자체 오류가 아니라 official image가 요구하는 runtime contract를 충족하지 않은 것이다.

DB readiness는 container가 Up인 것만으로 판단하지 않는다. docker ps는 process lifecycle을 보여주고, pg\_isready는 DB가 connection을 받을 준비가 되었는지 확인한다. SQL 실행은 실제 database access evidence다.

## 실행 명령

```
docker network create paperclip-day3-net
docker volume create paperclip-day3-pgdata

docker run -d \
  --name paperclip-day3-postgres \
  --network paperclip-day3-net \
  -e POSTGRES_PASSWORD=paperclip \
  -e POSTGRES_DB=paperclip \
  -v paperclip-day3-pgdata:/var/lib/postgresql/data \
  postgres:16-alpine
```

## 확인 명령

```
docker ps --filter name=paperclip-day3-postgres
docker logs paperclip-day3-postgres
docker exec paperclip-day3-postgres pg_isready -U postgres
docker exec paperclip-day3-postgres psql -U postgres -d paperclip -c "select current_database();"
docker run --rm --network paperclip-day3-net postgres:16-alpine pg_isready -h paperclip-day3-postgres -U postgres
```

## Linux 사전 테스트 결과

docker ps:

```
postgres:16-alpine
Up
5432/tcp
```

logs 핵심 줄:

```
database system is ready to accept connections
```

readiness:

```
/var/run/postgresql:5432 - accepting connections
paperclip-day3-postgres:5432 - accepting connections
```

SQL:

```
current_database
paperclip
```

## 실행 옵션 해석

| 옵션                                                | 역할                                 |
|---------------------------------------------------|------------------------------------|
| --network paperclip-day3-net                      | 같은 network container가 name으로 접근 가능 |
| -e POSTGRES_PASSWORD=paperclip                    | superuser password 초기화             |
| -e POSTGRES_DB=paperclip                          | 초기 database 생성                     |
| -v paperclip-day3-pgdata:/var/lib/postgresql/data | DB data persistence                |
| image postgres:16-alpine                          | DB runtime base                    |

## 핵심 유의사항

DB container의 5432/tcp가 보인다고 host에서 localhost:5432로 접속할 수 있다는 뜻은 아니다. host publish가 없으면 host에 port가 열리지 않는다. Day 3에서는 internal network 통신을 보기 위해 host publish를 생략한다.

PostgreSQL 초기화 로그는 길다. 모든 줄을 읽을 필요는 없지만 database system is ready to accept connections는 readiness 판단에 중요하다.

pg\_isready가 성공해도 application query가 항상 성공한다는 뜻은 아니다. DB process readiness와 schema/application readiness는 다르다. 초급 단계에서는 process readiness와 SQL 한 줄을 분리해서 확인한다.

## 자주 놓치는 지점

| 놓치는 지점             | 증상                 | 확인                |             |
|--------------------|--------------------|-------------------|-------------|
| password env 누락    | init error         | POSTGRES_PASSWORD |             |
| volume 재사용         | DB 초기화 값이 달라지지 않음  | volume inspect    |             |
| readiness 전 SQL 실행 | connection refused | pg_isready        |             |
| host publish 기대    | localhost 접속 실패    | docker ps PORTS   |             |
| logs 전체를 놓침        | 준비 완료 줄을 못 찾음      | `docker logs`     | grep ready` |

## 운영 관점

DB는 "떠 있다"보다 "준비되었다"가 중요하다. container process가 살아 있어도 DB가 connection을 받을 준비가 안 됐을 수 있다. readiness check는 application startup 순서, health check, Compose dependency에서 중요한 개념으로 이어진다.

## 확장 실습: host publish 없이 DB 확인

Day 3 DB 실습은 -p를 쓰지 않는다. 그래도 같은 Docker network 안의 container에서는 DB에 접근할 수 있다.

```
docker run --rm \
  --network paperclip-day3-net \
  postgres:16-alpine \
  pg_isready -h paperclip-day3-postgres -U postgres
```

Linux 사전 테스트 결과:

```
paperclip-day3-postgres:5432 - accepting connections
```

해석:

- DB가 host에 공개되지 않아도 internal network 통신은 가능하다.
- web app container는 Day 4에서 DB\_HOST=postgres 같은 service name으로 DB를 찾게 된다.
- 보안상 외부 공개가 필요 없는 service는 host publish를 피하는 것이 좋다.

## logs에서 볼 핵심 줄

| 로그 조각                                          | 의미                  |
|------------------------------------------------|---------------------|
| starting PostgreSQL 16.13                      | DB process 시작       |
| listening on IPv4 address "0.0.0.0", port 5432 | container 내부 listen |
| database system is ready to accept connections | readiness           |
| CREATE DATABASE                                | POSTGRES_DB 초기화     |
| superuser password is not specified            | 필수 env 누락           |

## 기록 템플릿

```
## Lesson 6 DB Evidence
- network:
- volume:
- container:
- env:
- image:
- readiness:
- SQL result:
- logs 핵심 줄:
- host publish 여부:
```

## 마무리 점검

PostgreSQL 초기화에 필요한 password env는 \_\_\_\_이다.  
 DB readiness는 `docker ps`만으로 판단하지 않고 \_\_\_\_로 확인한다.  
 host publish가 없으면 host의 `localhost:5432` 접근은 \_\_\_\_ 수 있다.

## cleanup

```
docker rm -f paperclip-day3-postgres
docker volume rm paperclip-day3-pgdata
docker network rm paperclip-day3-net
```

## 평가 기준

| 기준        | 2점 evidence                       |
|-----------|-----------------------------------|
| 실행 조건     | env/network/volume을 함께 지정했다       |
| readiness | pg_isready를 확인했다                  |
| SQL       | database query 결과를 확보했다           |
| 해석        | host publish와 internal port를 구분했다 |

## 공식 근거 링크

- PostgreSQL Docker Official Image: [https://hub.docker.com/\\_/postgres](https://hub.docker.com/_/postgres)
- Docker run reference: <https://docs.docker.com/reference/cli/docker/container/run/>